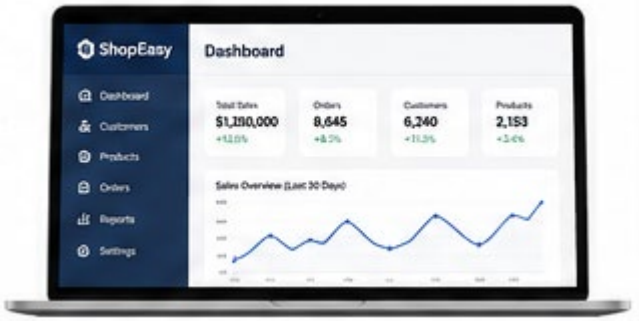
NUS National University of Singapore

BT5110
Data Management and Warehousing

Relational Model

ShopEasy: An E-Commerce Company

- Imagine you are the Data Team at ShopEasy, a fast-growing e-commerce startup
- The CEO wants a new system to replace Excel spreadsheets
- **Your mission:**
  - Design the database
  - Support web & mobile apps
  - Enable business analytics





What Data does the Application Need?

- What information must the system remember?



- These are the **key entities** in the business.
- We will store and organize them in a database

The CEO's Questions

- The CEO asks these questions every morning
  - ✓ Who are our top 20 customers?
  - ✓ Which product are selling fastest?
  - ✓ Which suppliers are delayed?
  - ✓ How much revenue did we make yesterday?
  - ✓ Which customers have not purchased in 6 months?



Can Excel answer these quickly if we have 10 million orders?

The Spreadsheet Disaster

OrderID	OrderDate	CustName	Phone	ProductName	Price	Qty	Salesperson
1001	1-Jul	Alice Tan	91234567	Laptop	1800	1	John
1001	1-Jul	Alice Tan	91234567	Mouse	25	2	John
1002	2-Jul	Bob Lee	87654321	Monitor	350	1	Mary
1002	2-Jul	Bob Lee	87654321	Keyboard	45	1	Mary
1003	3-Jul	Alice Tan	91234567	Monitor	350	1	John

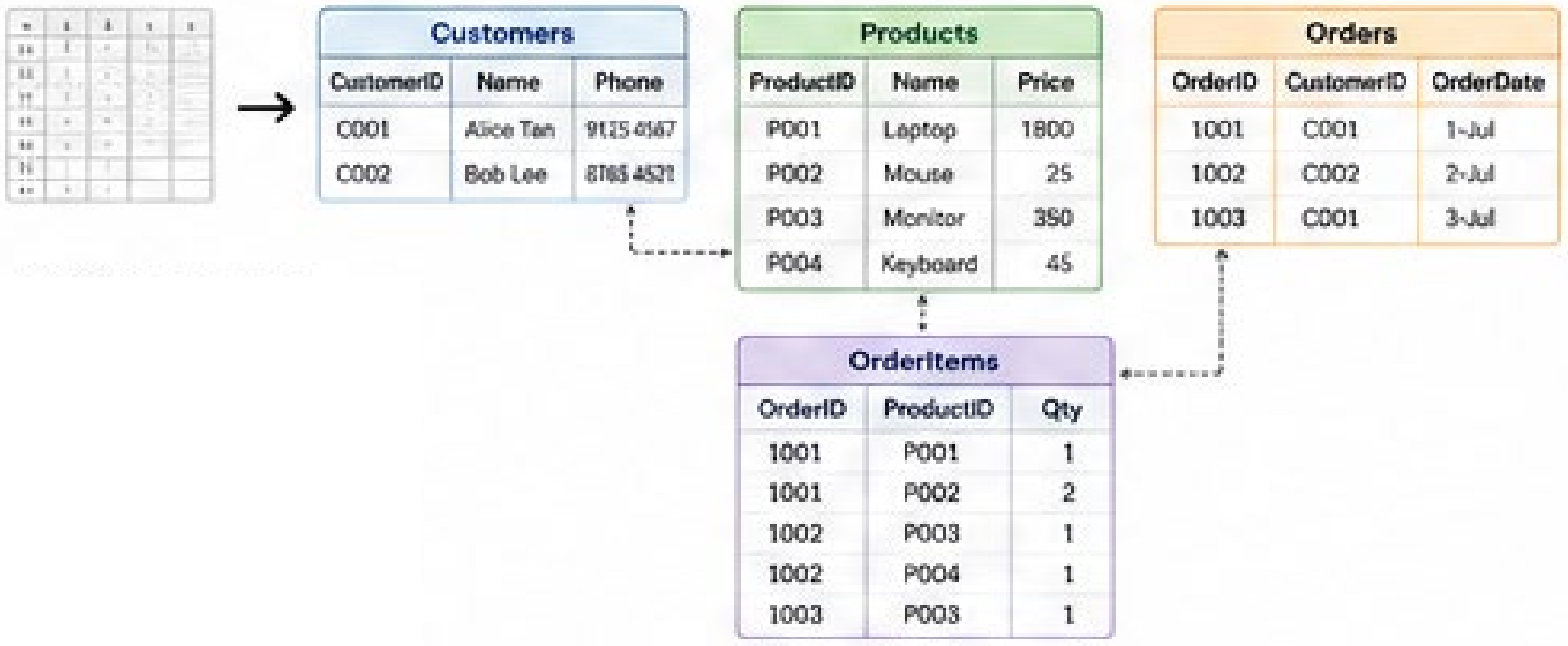
- **Problems:**
  - Customer info repeated
  - Prices repeated
  - Hard to update
  - Deleting a row may lose important info
  - Difficult to answer questions

Update Anomaly

If Alice changes her phone number, how many rows must be updated?

The Turning Point: From Spreadsheet to Relational Model

- Split the big table into related tables



Customers

CustomerID	Name	Phone
C001	Alice Tan	9123 4567
C002	Bob Lee	8765 4321

Products

ProductID	Name	Price
P001	Laptop	1800
P002	Mouse	25
P003	Monitor	350
P004	Keyboard	45

Orders

OrderID	CustomerID	OrderDate
1001	C001	1-Jul
1002	C002	2-Jul
1003	C001	3-Jul

OrderItems

OrderID	ProductID	Qty
1001	P001	1
1001	P002	2
1002	P003	1
1002	P004	1
1003	P003	1

- This idea changed computing forever.
- It is called the **Relational Model**

Relational Model Basics

- The Relational Model was introduced by Edgar F. Codd from IBM Research Lab in 1970
- Data is modeled in terms of **relations**, tables with rows and columns

Table "Employees"

id	name	dob	salary
1	Alice	10-08-1988	7,500
2	Bob	06-11-2001	4,800
3	Carol	25-02-1995	5,500

- Relations are defined using **schema** specifying **attributes** and **constraints**
  - Employees (id: **INT**, name: **TEXT**, dob: **DATE**, salary: **NUMERIC**)

Relational Model Basics

- **Domain:** set of atomic values
  - $dom(A_i)$ is the domain of **attribute** A_i (set of valid values)
  - Each value v of attribute A_i can also be **`null`**

null means value is unknown or missing

Table "Employees"

id	name	dob	salary
1	Alice	10-08-1988	7,500
2	Bob	06-11-2001	4,800
3	Carol	25-02-1995	5,500

- **Relation:** set of **tuples** (or records, or rows)
  - $R(A_1, A_2, \ldots, A_n)$ is a schema named R with n attributes
  - Each **instance** of a schema R is a relation (or table) that is a subset of $\{(a_1, a_2, \ldots, a_n) \mid a_i \in dom(A_i) \cup \{null\}\}$

Example

- **Relation** Course(code, mc, exam)
  - dom(code) = {cs2102, bt5110}
  - dom(mc) = {2, 4}
  - dom(exam) = {yes, no}

- **Instance**
  - Subset of the Cartesian product of the domains

    {cs2102, bt5110, `null`} ×
    {2, 4, `null`} ×
    {yes, no, `null`}

course	mc	exam
cs2102	2	yes
cs2102	2	no
cs2102	4	yes
cs2102	4	no
bt5110	2	yes
...	...	...
`null`	4	no
`null`	`null`	no
`null`	`null`	`null`

From Relations to Databases

- **Database Schema:** set of schema + constraints
  - Movies (id: **INT**, title: **TEXT**, genre: **TEXT**, releaseYear: **INT**)
  - Casts (movie: **INT**, actor: **INT**, role: **TEXT**)
  - Actors (id: **INT**, name: **TEXT**, dob: **DATE**)
- **Database:** collection of **related tables**

Table "Movies"

id	title	genre	releaseYear
101	Aliens	action	1986
101	Logan	drama	2017
103	Heat	crime	1995
104	Terminator	action	1984

Table "Casts"

movie	actor	role
101	20	Ellen Ripley
101	23	Private Hudson
101	54	Corporal Hicks
102	21	Logan
abc	23	Punk Leader

Table "Actors"

id	name	dob
20	Sigourney Weaver	08-10-2049
21	Hugh Jackman	12-10-1968
null	Tom Hanks	09-07-1956
23	Bill Paxton	17-05-1955

Spot the Problems

Table "Movies"

id	title	genre	releaseYear
101	Aliens	action	1986
101	Logan	drama	2017
103	Heat	crime	1995
104	Terminator	action	1984

Table "Casts"

movie	actor	role
101	20	Ellen Ripley
101	23	Private Hudson
101	54	Corporal Hicks
102	21	Logan
abc	23	Punk Leader

Table "Actors"

id	name	dob
20	Sigourney Weaver	08-10-2049
21	Hugh Jackman	12-10-1968
null	Tom Hanks	09-07-1956
23	Bill Paxton	17-05-1955

Integrity Constraints

- Ensure that only valid data is stored in the database
- **Domain constraints**: Specifying valid values
- **Structural constraints**:
  - Primary key constraint (uniqueness)
  - Foreign key constraint (existence)
- General constraints: Check and Triggers

Table "Movies"

id	title	genre	releaseYear
101	**Aliens**	action	1986
101	**Logan**	drama	2017
103	Heat	crime	1995
104	Terminator	action	1984

Table "Casts"

movie	actor	role
101	20	Ellen Ripley
101	23	Private Hudson
101	**54**	Corporal Hicks
102	21	Logan
abc	23	Punk Leader

Table "Actors"

id	name	dob
20	Sigourney Weaver	**08-10-2049**
21	Hugh Jackman	12-10-1968
null	Tom Hanks	09-07-1956
23	Bill Paxton	17-05-1955

Creating Tables and Inserting Data

- **Database Schema:** set of schema + constraints
  - Movies (id: **INT**, title: **TEXT**, genre: **TEXT**, releaseYr: **INT**)
- **SQL**

```
CREATE TABLE Movies (
  id INT,
  title TEXT,
  genre TEXT,
  releaseYr INT
);
```

Table "Movies"

id	title	genre	releaseYr
101	Aliens	action	1986
101	Logan	drama	2017
103	Heat	crime	1995
104	Terminator	action	1984

```
INSERT INTO Movies VALUES (101, 'Aliens', 'action', 1986);
```

Would this SQL statement prevent duplicate movie IDs?

Would it prevent NULL movie IDs?

Creating Tables and Inserting Data

- **Database Schema:** set of schema + constraints
  - Movies (id: **INT**, title: **TEXT**, genre: **TEXT**, releaseYr: **INT**)
- **SQL**

```
CREATE TABLE Movies (
  id INT PRIMARY KEY,
  title TEXT NOT NULL,
  genre TEXT,
  releaseYr INT
);
```

- **Now the database will reject duplicate movie IDs and prevent missing titles**

Creating Tables and Inserting Data

- **Database Schema:** set of schema + constraints
  - Casts (movie: **INT**, actor: **INT**, role: **TEXT**)
- **SQL**

```
CREATE TABLE Casts (
  movie   INT,
  actor   INT,
  role    VARCHAR(64)
);
```

Table "Casts"

movie	actor	role
101	20	Ellen Ripley
101	23	Private Hudson
101	54	Corporal Hicks
102	21	Logan
abc	23	Punk Leader

- **Only data of the correct type can be inserted**

```
INSERT INTO Casts VALUES (101, 20, 'Ellen Ripley');
INSERT INTO Casts VALUES ('abc', 23, 'Punk Leader');
```

Complete list of built-in data types of PostgreSQL: https://www.postgresql.org/docs/current/datatype.html

Creating Tables and Inserting Data

- **Database Schema:** set of schema + constraints
  - Casts (movie: **INT**, actor: **INT**, role: **TEXT**)
- **SQL**

```
CREATE TABLE Casts (
  movie   INT,
  actor   INT,
  role    VARCHAR(64),
  FOREIGN KEY (movie) REFERENCES Movies(id),
  FOREIGN KEY (actor) REFERENCES Actors(id)
);
```

- **Now the database will check if the movie exists and if the actor exists before inserting a cast record**

Creating Tables and Inserting Data

- **Database Schema:** set of schema + constraints
  - Actors (id: **INT**, name: **TEXT**, dob: **DATE**)
- **SQL**

```
CREATE TABLE Actors (
  id    INT,
  name  TEXT,
  dob   DATE
);
```

Table "Actors"

id	name	dob
20	Sigourney Weaver	08-10-2049
21	Hugh Jackman	12-10-1968
null	Tom Hanks	09-07-1956

- **We can specify the attributes explicitly**
  - `INSERT INTO Actors (id, name, dob)`
    `VALUES (20, 'Sigourney Weaver', '2049-10-08');`
  - `INSERT INTO Actors (name, dob)`
    `VALUES ('Tom Hanks', '1956-07-09');`

Creating Tables and Inserting Data

- **Database Schema:** set of schema + constraints
  - Actors (id: **INT**, name: **TEXT**, dob: **DATE**)
- **SQL**

```
CREATE TABLE Actors (
  id    INT PRIMARY KEY,
  name  TEXT NOT NULL,
  dob   DATE
);
```

- **Now this INSERT statement will fail because it does not provide an ID**
  - `INSERT INTO Actors (name, dob)`
    `VALUES ('Tom Hanks', '1956-07-09');`

Key Constraints

- **Superkey:** any subset of attributes that *uniquely* identifies a row

- **Key:** a *minimal* superkey
  - If you remove any attribute, it no longer uniquely identifies the row
  - Extra attributes are redundant, don't help identify the row

Table "Movies"

id	title	genre	releaseYr
101	Aliens	action	1986
101	Logan	drama	2017
103	Heat	crime	1995
104	Terminator	action	1984

Attribute Set	Superkey?	Key?
{title}	✓	✓
{title, genre}	✓	✗
{title, genre, releaseYr}	✓	✗

Key Constraints

- **Candidate Keys:** set of all keys

- **Primary Key:** selected candidate key to be table's main identifier
  - We underline the primary key
  - Keys should not have `null` values

Table "Movies"

id	title	genre	releaseYr
101	Aliens	action	1986
101	Logan	drama	2017
103	Heat	crime	1995
104	Terminator	action	1984

Movies (id: **INT**, title: **TEXT**, genre: **TEXT,** ReleaseYr: **INT**)

- **Can ID be a primary key in this table?**

Key Constraints

Movies (id: **INT**, title: **TEXT**, genre: **TEXT**, releaseYr: **INT**)

Column Constraints

```
CREATE TABLE Movies (
  id       INT PRIMARY KEY,
  title    TEXT,
  genre    TEXT,
  releaseYr INT
);
```

Table Constraints

```
CREATE TABLE Movies (
  id       INT,
  title    TEXT,
  genre    TEXT,
  releaseYr INT,
  PRIMARY KEY (id)
);
```

- **Primary key prevents insertion of duplicate entries**

Key Constraints

Cast (movie: **INT**, actor: **INT**, role: TEXT)

Table "Casts"

movie	actor	role
101	20	Ellen Ripley
101	23	Private Hudson
101	54	Corporal Hicks
102	21	Logan
abc	23	Punk Leader

Column Constraints

```
CREATE TABLE casts (
  movie   INT PRIMARY KEY,
  actor   INT PRIMARY KEY,
  role    TEXT
);
```

Table Constraints

```
CREATE TABLE casts (
  movie   INT,
  actor   INT,
  role    TEXT,
  PRIMARY KEY (movie, actor)
);
```

- **A table can only have at most one primary key**

Key Constraints

Actors (id: **INT**, name: **TEXT**, dob: **DATE**)

Table "Actors"

id	name	dob
20	Sigourney Weaver	08-10-2049
21	Hugh Jackman	12-10-1968
~~null~~	~~Tom Hanks~~	~~09-07-1956~~
23	Bill Paxton	17-05-1955

```
CREATE TABLE actors (
  id      INT,
  name    TEXT,
  dob     DATE,
  PRIMARY KEY (id)
);
```

```
CREATE TABLE actors (
  id      INT NOT NULL,
  name    TEXT,
  dob     DATE,
  UNIQUE (id)
);
```

- **Primary key also prevents null values**
- **Primary key is equivalent to UNIQUE and NOT NULL**

Foreign Key Constraints

- **Foreign key:** Attribute whose values come from the primary key of another table
- **Referential Integrity:** Every foreign key value in a **referencing relation** must match an existing primary key value in the **referenced relation**

Table "Movies"

id	title	genre	releaseYr
101	Aliens	action	1986
102	Logan	drama	2017
103	Heat	crime	1995
104	Terminator	action	1984

referenced relation

Table "Casts"

movie	actor	role
101	20	Ellen Ripley
101	23	Private Hudson
101	**54**	Corporal Hicks
102	21	Logan
104	23	Punk Leader

referencing relation

Table "Actors"

id	name	dob
20	Sigourney Weaver	08-10-2049
21	Hugh Jackman	12-10-1968
22	Tom Hanks	09-07-1956
23	Bill Paxton	17-05-1955

referenced relation

Foreign Key Constraints

```
CREATE TABLE Movies (
  id      INT,
  title   TEXT,
  genre   TEXT,
  releaseYr INT,
  PRIMARY KEY (id)
);
```

```
CREATE TABLE Casts (
  movie INT
    REFERENCES Movies(id),
  actor INT
    REFERENCES Actors(id),
  role  TEXT,
  PRIMARY KEY (movie, actor)
);
```

```
CREATE TABLE Actors (
  id      INT,
  name    TEXT,
  dob     DATE,
  PRIMARY KEY (id)
);
```

Table "Movies"

id	title	genre	releaseYr
101	Aliens	action	1986
102	Logan	drama	2017
103	Heat	crime	1995
104	Terminator	action	1984

Table "Casts"

movie	actor	role
101	20	Ellen Ripley
101	23	Private Hudson
101	54	Corporal Hicks
102	21	Logan
104	23	Punk Leader

Table "Actors"

id	name	dob
20	Sigourney Weaver	08-10-2049
21	Hugh Jackman	12-10-1968
22	Tom Hanks	09-07-1956
23	Bill Paxton	17-05-1955

- **Now this statement will fail because it violates foreign key constraint**
  - **INSERT INTO** Casts **VALUES** (101, 54, 'Corporal Hicks');

Foreign Key Constraints

- Two equivalent ways to declare foreign keys

```
CREATE TABLE Movies (
  id      INT,
  title   TEXT,
  genre   TEXT,
  releaseYr INT,
  PRIMARY KEY (id)
);
```

```
CREATE TABLE Casts (
  movie INT
    REFERENCES Movies(id),
  actor INT,
  role  TEXT,
  FOREIGN KEY (actor)
    REFERENCES Actors(id),
  PRIMARY KEY (movie, actor)
);
```

```
CREATE TABLE Actors (
  id      INT,
  name    TEXT,
  dob     DATE,
  PRIMARY KEY (id)
);
```

Foreign Keys and Deleting Data

- Foreign key may **prevent deletion** of data in the **referenced relation**

```
DELETE FROM Actors
WHERE id = 21;
```



ERROR:
update or delete on table “Actors" violates foreign key constraint

Table "Movies"

id	title	genre	releaseYr
101	Aliens	action	1986
102	Logan	drama	2017
103	Heat	crime	1995
104	Terminator	action	1984

Table "Casts"

movie	actor	role
101	20	Ellen Ripley
101	23	Private Hudson
101	54	Corporal Hicks
102	21	Logan
104	23	Punk Leader

Table "Actors"

id	name	dob
20	Sigourney Weaver	08-10-2049
21	Hugh Jackman	12-10-1968
22	Tom Hanks	09-07-1956
23	Bill Paxton	17-05-1955

Foreign Keys and Deleting Data

```
CREATE TABLE Casts (
  movie INT REFERENCES Movies(id)
    ON DELETE CASCADE,
  actor INT,
  role  TEXT,
  FOREIGN KEY (actor) REFERENCES Actors(id)
    ON DELETE CASCADE,
  PRIMARY KEY (movie, actor)
);
```

Table "Movies"

id	title	genre	releaseYr
101	Aliens	action	1986
102	Logan	drama	2017
103	Heat	crime	1995
104	Terminator	action	1984

Table "Casts"

movie	actor	role
101	20	Ellen Ripley
101	23	Private Hudson
101	54	Corporal Hicks
102	21	Logan
104	23	Punk Leader

Table "Actors"

id	name	dob
20	Sigourney Weaver	08-10-2049
21	Hugh Jackman	12-10-1968
22	Tom Hanks	09-07-1956
23	Bill Paxton	17-05-1955

Check Constraints

- **Basic General Constraints:** Specify a Boolean condition that column values must satisfy

Actors (id: **INT**, name: **TEXT**, dob: **DATE**)

Table "Actors"

id	name	dob
20	Sigourney Weaver	08-10-2049
21	Hugh Jackman	12-10-1968
22	Tom Hanks	09-07-1956
23	Bill Paxton	17-05-1955

```
CREATE TABLE Actors (
  id      INT PRIMARY KEY,
  name    TEXT,
  dob     DATE
    CHECK (dob <= CURRENT_DATE)
);
```

- **Now this action will be rejected**
  - `INSERT INTO Actors VALUES (30, 'XXX', '07-09-2049');`

Check Constraints

- **Basic General Constraints:** Specify a Boolean condition that **multiple column values within the same row** must satisfy

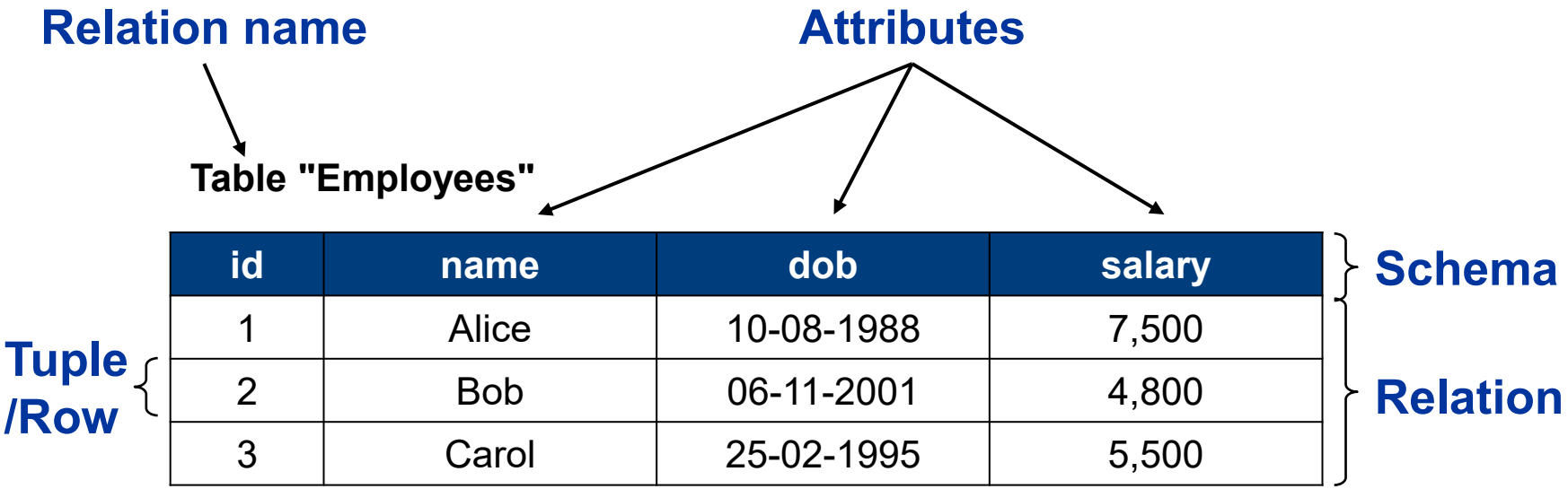
```
CREATE TABLE student (
  matric  CHAR(9) PRIMARY KEY,
  name    TEXT,
  enrol   DATE,
  grad    DATE CHECK (grad > enrol)
);
```

Check Constraints

- **Basic General Constraints:** Specify a Boolean condition that **multiple column values within the same row** must satisfy
- Can **combine multiple conditions** using logical operators such as AND, OR, and NOT

```
CREATE TABLE book (
  isbn    CHAR(13) PRIMARY KEY,
  title   TEXT,
  price   NUMERIC,
  edition VARCHAR(10)
    CHECK ((edition = ‘Hardcover' AND price > 100)
        OR (edition = 'Softcover' AND price > 0))
);
```

Summary


Relation name
Attributes
Table "Employees"
id
name
dob
salary
1
Alice
10-08-1988
7,500
2
Bob
06-11-2001
4,800
3
Carol
25-02-1995
5,500
Tuple /Row
Schema
Relation

Summary

- **Create Table**

```
CREATE TABLE table_name (
  attribute  type  <column constraint>,
    :
  attribute  type  <column constraint>,
  <table constraint>
);
```

- **Constraints**
  - `PRIMARY KEY` ; `UNIQUE` ; `CHECK`
  - `REFERENCES` (need `FOREIGN KEY` for table constraint)

Summary

- **Insert Data**

```
INSERT INTO table_name VALUES (<value>);
INSERT INTO table_name (<column>) VALUES (<value>);
```

- **Delete Data**

```
DELETE FROM table_name WHERE <condition>;
```

- **Update Data**

```
UPDATE table_name SET <assign> WHERE <condition>;
```

- **Drop Table**

```
DROP TABLE table_name;
```